

# ETL Script Documentation Report

## Script: etl\_customer\_360.sql

### Parsed Information

Type: SQL

Sources: customers, raw\_transactions

Targets: dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv

Transformations: group by, CTE/subquery, window function, order by, string transform, aggregation: count, aggregation: sum, join, aggregation: avg, partition, case/conditional, aggregation: max, filter/condition, deduplication

### Auto-Generated Documentation

#### **\*\*Overview\*\***

etl\_customer\_360.sql is a SQL ETL script that reads data from customers, raw\_transactions, applies transformations, and loads results into dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv.

#### **\*\*What It Does\*\***

- Reads from: customers, raw\_transactions
- Applies: group by, CTE/subquery, window function, order by, string transform, aggregation: count, aggregation: sum, join, aggregation: avg, partition, case/conditional, aggregation: max, filter/condition, deduplication
- Writes to: dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv

#### **\*\*Technical Summary\*\***

- Script Type: SQL
- Input(s): customers, raw\_transactions
- Output(s): dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv
- Operations: group by, CTE/subquery, window function, order by, string transform, aggregation: count, aggregation: sum, join, aggregation: avg, partition, case/conditional, aggregation: max, filter/condition, deduplication

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from customers, raw\_transactions into dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv, eliminating manual data handling and ensuring data is always current.

#### **\*\*Who Benefits\*\***

Data and analytics teams who depend on dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv for reporting, dashboards, and business decisions.

## ETL Script Documentation Report

### **\*\*Risk if Fails\*\***

If etl\_customer\_360.sql fails, dim\_customer\_segments, stg\_clean\_transactions, rpt\_customer\_360, stg\_customer\_ltv will not be updated ? causing stale or missing data in all downstream reports.

### **Impact Analysis**

Risk Level: MEDIUM

Reason: Failure affects 4 output(s) but no other scripts directly.

Depends on: ['customers', 'raw\_transactions']

Writes to: ['dim\_customer\_segments', 'stg\_clean\_transactions', 'rpt\_customer\_360', 'stg\_customer\_ltv']

Affected scripts if fails: None

# ETL Script Documentation Report

## Script: etl\_finance\_summary.sql

### Parsed Information

Type: SQL

Sources: raw\_expenses, departments

Targets: stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual

Transformations: group by, window function, order by, aggregation: min, aggregation: count, join, aggregation: sum, aggregation: avg, null handling, left join, string transform, numeric transform, case/conditional, aggregation: max, filter/condition, deduplication

### Auto-Generated Documentation

#### **\*\*Overview\*\***

etl\_finance\_summary.sql is a SQL ETL script that reads data from raw\_expenses, departments, applies transformations, and loads results into stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual.

#### **\*\*What It Does\*\***

- Reads from: raw\_expenses, departments
- Applies: group by, window function, order by, aggregation: min, aggregation: count, join, aggregation: sum, aggregation: avg, null handling, left join, string transform, numeric transform, case/conditional, aggregation: max, filter/condition, deduplication
- Writes to: stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual

#### **\*\*Technical Summary\*\***

- Script Type: SQL
- Input(s): raw\_expenses, departments
- Output(s): stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual
- Operations: group by, window function, order by, aggregation: min, aggregation: count, join, aggregation: sum, aggregation: avg, null handling, left join, string transform, numeric transform, case/conditional, aggregation: max, filter/condition, deduplication

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from raw\_expenses, departments into stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual, eliminating manual data handling and ensuring data is always current.

## ETL Script Documentation Report

### **\*\*Who Benefits\*\***

Data and analytics teams who depend on stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual for reporting, dashboards, and business decisions.

### **\*\*Risk if Fails\*\***

If etl\_finance\_summary.sql fails, stg\_departments, fact\_vendor\_spend, stg\_raw\_expenses, fact\_monthly\_expenses, rpt\_budget\_vs\_actual will not be updated ? causing stale or missing data in all downstream reports.

### **Impact Analysis**

Risk Level: MEDIUM

Reason: Failure affects 5 output(s) but no other scripts directly.

Depends on: ['raw\_expenses', 'departments']

Writes to: ['stg\_departments', 'fact\_vendor\_spend', 'stg\_raw\_expenses', 'fact\_monthly\_expenses', 'rpt\_budget\_vs\_actual']

Affected scripts if fails: None

# ETL Script Documentation Report

## Script: etl\_hr\_payroll.py

### Parsed Information

Type: PYTHON

Sources: data/attendance.csv, data/employees.csv, data/deductions.xlsx

Targets: hr.db

Transformations: sort\_values, new column: gross\_pay, merge, new column: net\_pay, apply, new column: attendance\_pct, drop\_duplicates, new column: deduction\_amount, filter/condition, fillna

### Auto-Generated Documentation

#### **\*\*Overview\*\***

etl\_hr\_payroll.py is a PYTHON ETL script that reads data from data/attendance.csv, data/employees.csv, data/deductions.xlsx, applies transformations, and loads results into hr.db.

#### **\*\*What It Does\*\***

- Reads from: data/attendance.csv, data/employees.csv, data/deductions.xlsx
- Applies: sort\_values, new column: gross\_pay, merge, new column: net\_pay, apply, new column: attendance\_pct, drop\_duplicates, new column: deduction\_amount, filter/condition, fillna
- Writes to: hr.db

#### **\*\*Technical Summary\*\***

- Script Type: PYTHON
- Input(s): data/attendance.csv, data/employees.csv, data/deductions.xlsx
- Output(s): hr.db
- Operations: sort\_values, new column: gross\_pay, merge, new column: net\_pay, apply, new column: attendance\_pct, drop\_duplicates, new column: deduction\_amount, filter/condition, fillna

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from data/attendance.csv, data/employees.csv, data/deductions.xlsx into hr.db, eliminating manual data handling and ensuring data is always current.

#### **\*\*Who Benefits\*\***

Data and analytics teams who depend on hr.db for reporting, dashboards, and business decisions.

#### **\*\*Risk if Fails\*\***

## ETL Script Documentation Report

If etl\_hr\_payroll.py fails, hr.db will not be updated ? causing stale or missing data in all downstream reports.

### Impact Analysis

Risk Level: MEDIUM

Reason: Failure affects 1 output(s) but no other scripts directly.

Depends on: ['data/attendance.csv', 'data/employees.csv', 'data/deductions.xlsx']

Writes to: ['hr.db']

Affected scripts if fails: None

# ETL Script Documentation Report

## Script: etl\_product\_analytics.py

### Parsed Information

Type: PYTHON

Sources: data/orders.csv, data/returns.csv, data/products.csv, data/categories.csv

Targets: N/A

Transformations: new column: revenue, agg, merge, reset\_index, apply, new column: return\_rate, new column: performance\_grade, new column: is\_returned, groupby, filter/condition

### Auto-Generated Documentation

#### **\*\*Overview\*\***

etl\_product\_analytics.py is a PYTHON ETL script that reads data from data/orders.csv, data/returns.csv, data/products.csv, data/categories.csv, applies transformations, and loads results into N/A.

#### **\*\*What It Does\*\***

- Reads from: data/orders.csv, data/returns.csv, data/products.csv, data/categories.csv
- Applies: new column: revenue, agg, merge, reset\_index, apply, new column: return\_rate, new column: performance\_grade, new column: is\_returned, groupby, filter/condition
- Writes to: N/A

#### **\*\*Technical Summary\*\***

- Script Type: PYTHON
- Input(s): data/orders.csv, data/returns.csv, data/products.csv, data/categories.csv
- Output(s): N/A
- Operations: new column: revenue, agg, merge, reset\_index, apply, new column: return\_rate, new column: performance\_grade, new column: is\_returned, groupby, filter/condition

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from data/orders.csv, data/returns.csv, data/products.csv, data/categories.csv into unknown targets, eliminating manual data handling and ensuring data is always current.

#### **\*\*Who Benefits\*\***

Data and analytics teams who depend on unknown targets for reporting, dashboards, and business decisions.

#### **\*\*Risk if Fails\*\***

## ETL Script Documentation Report

If etl\_product\_analytics.py fails, unknown targets will not be updated ? causing stale or missing data in all downstream reports.

### Impact Analysis

Risk Level: LOW

Reason: This script has no downstream dependencies.

Depends on: ['data/orders.csv', 'data/returns.csv', 'data/products.csv', 'data/categories.csv']

Writes to: None

Affected scripts if fails: None



# ETL Script Documentation Report

## Script: etl\_sales\_pipeline.py

### Parsed Information

Type: PYTHON

Sources: data/sales\_transactions.csv, data/customers.csv

Targets: sales\_summary

Transformations: new column: revenue, merge, apply, new column: sale\_year, dropna, new column: sale\_month, new column: net\_revenue, new column: discount\_applied, new column: customer\_segment, filter/condition, fillna

### Auto-Generated Documentation

#### **\*\*Overview\*\***

etl\_sales\_pipeline.py is a PYTHON ETL script that reads data from data/sales\_transactions.csv, data/customers.csv, applies transformations, and loads results into sales\_summary.

#### **\*\*What It Does\*\***

- Reads from: data/sales\_transactions.csv, data/customers.csv
- Applies: new column: revenue, merge, apply, new column: sale\_year, dropna, new column: sale\_month, new column: net\_revenue, new column: discount\_applied, new column: customer\_segment, filter/condition, fillna
- Writes to: sales\_summary

#### **\*\*Technical Summary\*\***

- Script Type: PYTHON
- Input(s): data/sales\_transactions.csv, data/customers.csv
- Output(s): sales\_summary
- Operations: new column: revenue, merge, apply, new column: sale\_year, dropna, new column: sale\_month, new column: net\_revenue, new column: discount\_applied, new column: customer\_segment, filter/condition, fillna

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from data/sales\_transactions.csv, data/customers.csv into sales\_summary, eliminating manual data handling and ensuring data is always current.

#### **\*\*Who Benefits\*\***

Data and analytics teams who depend on sales\_summary for reporting, dashboards, and business decisions.

#### **\*\*Risk if Fails\*\***

## ETL Script Documentation Report

If etl\_sales\_pipeline.py fails, sales\_summary will not be updated ? causing stale or missing data in all downstream reports.

### Impact Analysis

Risk Level: MEDIUM

Reason: Failure affects 1 output(s) but no other scripts directly.

Depends on: ['data/sales\_transactions.csv', 'data/customers.csv']

Writes to: ['sales\_summary']

Affected scripts if fails: None

# ETL Script Documentation Report

**Script:** sample\_hr.py

## Parsed Information

Type: PYTHON

Sources: data/attendance.csv, data/employees.csv

Targets: hr\_report, database/etl\_docs.db

Transformations: merge, filter/condition, new column: bonus

## Auto-Generated Documentation

### **\*\*Overview\*\***

This script reads from data/attendance.csv, data/employees.csv and writes to database/etl\_docs.db, hr\_report.

### **\*\*What It Does\*\***

- Sources: data/attendance.csv, data/employees.csv
- Transformations: new column: bonus, filter/condition, merge

### **\*\*Output\*\***

Writes to: database/etl\_docs.db, hr\_report

## Business Purpose

### **\*\*Business Purpose\*\***

This script automates processing of data from data/attendance.csv, data/employees.csv into hr\_report, database/etl\_docs.db, eliminating manual data handling and ensuring data is always current.

### **\*\*Who Benefits\*\***

Data and analytics teams who depend on hr\_report, database/etl\_docs.db for reporting, dashboards, and business decisions.

### **\*\*Risk if Fails\*\***

If sample\_hr.py fails, hr\_report, database/etl\_docs.db will not be updated ? causing stale or missing data in all downstream reports.

## Impact Analysis

Risk Level: MEDIUM

Reason: Failure affects 2 output(s) but no other scripts directly.

Depends on: ['data/attendance.csv', 'data/employees.csv']

## ETL Script Documentation Report

Writes to: ['hr\_report', 'database/etl\_docs.db']

Affected scripts if fails: None

# ETL Script Documentation Report

## Script: sample\_orders.py

### Parsed Information

Type: PYTHON

Sources: data/orders.csv

Targets: big\_orders, database/etl\_docs.db

Transformations: rename, filter/condition, new column: processed

### Auto-Generated Documentation

#### **\*\*Overview\*\***

This script reads from data/orders.csv and writes to database/etl\_docs.db, big\_orders.

#### **\*\*What It Does\*\***

- Sources: data/orders.csv
- Transformations: new column: processed, rename, filter/condition

#### **\*\*Output\*\***

Writes to: database/etl\_docs.db, big\_orders

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from data/orders.csv into big\_orders, database/etl\_docs.db, eliminating manual data handling and ensuring data is always current.

#### **\*\*Who Benefits\*\***

Data and analytics teams who depend on big\_orders, database/etl\_docs.db for reporting, dashboards, and business decisions.

#### **\*\*Risk if Fails\*\***

If sample\_orders.py fails, big\_orders, database/etl\_docs.db will not be updated ? causing stale or missing data in all downstream reports.

### Impact Analysis

Risk Level: MEDIUM

Reason: Failure affects 2 output(s) but no other scripts directly.

Depends on: ['data/orders.csv']

## ETL Script Documentation Report

Writes to: ['big\_orders', 'database/etl\_docs.db']

Affected scripts if fails: None

# ETL Script Documentation Report

## Script: sample\_sales.sql

### Parsed Information

Type: SQL

Sources: customers, orders

Targets: sales\_summary

Transformations: group by, aggregation: count, aggregation: sum, join, filter/condition

### Auto-Generated Documentation

#### **\*\*Overview\*\***

This script reads from orders, customers and writes to sales\_summary.

#### **\*\*What It Does\*\***

- Sources: orders, customers
- Transformations: aggregation: count, aggregation: sum, group by, join, filter/condition

#### **\*\*Output\*\***

Writes to: sales\_summary

### Business Purpose

#### **\*\*Business Purpose\*\***

This script automates processing of data from customers, orders into sales\_summary, eliminating manual data handling and ensuring data is always current.

#### **\*\*Who Benefits\*\***

Data and analytics teams who depend on sales\_summary for reporting, dashboards, and business decisions.

#### **\*\*Risk if Fails\*\***

If sample\_sales.sql fails, sales\_summary will not be updated ? causing stale or missing data in all downstream reports.

### Impact Analysis

Risk Level: MEDIUM

Reason: Failure affects 1 output(s) but no other scripts directly.

Depends on: ['customers', 'orders']

Writes to: ['sales\_summary']

Affected scripts if fails: None